

One Bite Restaurant Management System

CONTENTS

- Program Overview.
- Folder Structure.
- Project Modules.
- Data Management.
- Classes.
- Requirements.
- System Setup.
- System Workflow.
- Conclusion.

Authors:

Anjali Nagarkoti

Gaurav Karki

Kumkum Kapkoti

Nisha Modi

Vimal Shahi

(Team 4, Batch 1, Full-Stack Web Development, Indixpert)

(2024-25)

Program Overview:

The **One Bite Restaurant Management System** is a Python application that streamlines restaurant operations, including user authentication, table booking, menu management, order processing, and billing. It allows admins and staff to manage roles, handle reservations, update menus, process orders, and generate bills with payment handling. Using JSON files for data storage, it operates efficiently with a database, enhancing restaurant management and customer service.

Folder Structure:

The system is organized into a modular structure:

- `src/authentication` : Handles user authentication and management.
- `src/booking` : Manages table bookings with time slots.
- `src/dashboard` : Provides all the dashboards.
- `src/database` : Stores data in JSON files for users, bookings, orders, etc.
- `src/menu` : Manages menu items and prices.
- `src/orders` : Processes customer orders and links them to bookings.
- `src/manage_bill` : Handles bill generation and payment processing.
- `src/payments` : Handles payment transactions.
- `src/utility` : Contains utilities for validation, logging, and color-coded output
- `main.py` : Entry point for the application.

Project Modules:

1. Authentication Module:

- **Purpose:** Manages user authentication and role-based access.
- **Features:**
 - User signup and login.
 - Admin and staff roles.
 - Secure password storage.

2. Table Booking Module:

- **Purpose:** Allows table reservations with time slots.
- **Features:**
 - Book tables with specific slots.
 - View available tables for a date.
 - Cancel bookings.

3. Menu Management:

- **Purpose:** Manage restaurant menu items.
- **Features:**
 - Add, edit, and delete menu items. (admin only)
 - Support for full and half-portion pricing.

4. Order Management:

- **Purpose:** Handle customer orders.
- **Features:**
 - Create, update, and cancel orders.
 - Associate orders with table bookings.
 - Support for takeout orders.

5. Billing and Payment:

- **Purpose:** Generate bills and handle payments.
- **Features:**
 - Generate bills based on orders.
 - Accept payments via cash or card.

6. Dashboards:

- **Admin Dashboard:**
 - Manage users, menu, and orders.
- **Staff Dashboard:**
 - Manage bookings, orders, and bills.

Data Management:

The system uses JSON files for data storage, that are stored in the database folder.

- **user.json** : Stores user credentials and roles.
- **booking.json** : Stores table booking information.
- **menu.json** : Stores menu items with prices.
- **order.json** : Stores customer orders and details.
- **bill.json** : Stores billing records.
- **payment.json** : Stores payment details.

Each file is loaded into the application at runtime and saved upon updates, ensuring persistent data storage.

Classes:

- **System (user_authentication.py)** : Manages user login, sign-up, and user management.
- **TableBookingSystem (table_booking.py)**: Handles viewing, booking, and canceling tables.
- **OrderFeature (order_feature.py)**: Manages orders, including adding, updating, and canceling.
- **BillFeature (bill_feature.py)**: Handles bill generation, updates, and payment processing.
- **Menu (menu.py)**: Loads, views, and manages menu items.

Requirements:

- Latest version of VSCode from Microsoft. Learn how to install VSCode [here](#).
- Version control software like Git. Learn how to install Git [here](#).
- Python version 3.12 or above. Learn how to install Python [here](#).

System Setup:

- **After installing all the requirements:**
- **Clone the Repository:**
 - Clone the repository into a location in your system.

```
C:\Users\asus>git clone https://github.com/Nishahtc/Indixpert-FSD-April-Python-Pro2-Team-4.git
```

- **Execution:**
 - In VSCode open a new python terminal and type “python main.py” to run the program.

System Workflow:

- **Launch Application:**
 - The system starts by displaying a menu for login or signup.
- **Authentication:**
 - Users login with role-specific access to dashboards.
- **Role-Based Access Control:**
 - **Admin users:** Full access to user management, menu updates, and reporting.
 - **Staff users:** Limited access to table bookings, orders, and bills.
- **Operational Workflows:**
 - **Table Booking:**
 - Staff can book tables for customers by selecting a date and time slot.
 - **Order Placement:**
 - Orders are linked to specific tables or created as takeout.
 - **Billing and Payment:**
 - Bills are generated for completed orders, and payments are processed.
- **Persistence:**
 - Data is stored in JSON files to ensure changes persist across sessions.

Conclusion:

In conclusion, The Restaurant Management System is a robust solution for managing restaurant operations efficiently. By using a file-based data storage approach, it eliminates the need for a database server, making it lightweight and easy to deploy.

By leveraging a modular design with JSON-based data storage, the system is lightweight, efficient, and easily deployable. It empowers restaurant administrators and staff to handle their tasks more effectively, reducing manual effort and enhancing overall service quality.